

# Stacks and Queues

Kanishk Goel

2 June, 2026

## 1 Minimum Stack

**Objective:** To modify stack such that we can find the minimum of the stack in  $O(1)$ .

**Modification:** Instead of an element, make a stack of pairs, where the second element is the minimum found till that element.

## 2 Minimum Queue

Remember head - Dequeue position, Tail - Enqueue position.

**Objective:** To modify queue such that we can find the minimum of queue in  $O(1)$ .

### 2.1 Method 1

Smallest value will always be stored at the head. This method doesn't store all the elements. The queue will be kept in nondecreasing order with the smallest value on the head.

#### 2.1.1 Adding elements

While adding elements to the tail, we require the element to be the maximum, therefore, we remove all the trailing elements bigger than the element we are adding first. This operation is allowed because the elements we removed will never be the minimum of the array.

#### 2.1.2 Removing elements

We check if the value at the head is equal to the value of the element we wish to remove. If it is, then we remove the element, otherwise we do nothing.

### 2.1.3 Runtime

Since every element is only pushed and/or popped once, the average runtime is  $O(1)$ .

## 2.2 Modification to Method 1

To not need to know which element is it we are removing, we can store the index for each element in the queue and remember how many elements we already have added and removed.

## 2.3 Method 2

A complicated implementation, but this way we can store all elements and also remove an element without knowing its value.

The idea is to simulate a queue using two stacks. We make two min stacks  $s1$  and  $s2$ . We will add new elements to  $s1$  and remove elements from  $s2$ . If  $s2$  is empty, we move all elements from  $s1$  to  $s2$ . Minimum of the queue here equals to minimum of both the stacks.

### 2.3.1 Runtime

$O(1)$  because each element is stored in  $s1$  once, transferred to  $s2$  once and once popped from  $s2$ .

## 2.4 Application

| Finding minimum for all subarrays of given length $M$                                                                   |
|-------------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> Array $A$ of length $N$ and $M$ such that $M \leq N$<br><b>Output:</b> Linear time solution for the same. |

**Solution:** Add first  $M$  elements of the array and output its minimum. Then add next element to the queue while removing the first element and so on.

### 3 Monotonic Stack

| Nearest smaller element                                                                                             |
|---------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> Integer array $A$ of size $n$<br><b>Output:</b> Rightmost index $j$ such that $j < i$ and $a_i > a_j$ |

Make a stack that stores value-index pairs. Iterate over the array from left to right, for some index  $i$ , compute the rightmost index for  $i$  as follows:

- Pop the top element of the stack as long as  $\text{value}_{stk.top} \geq a_i$  (this is because we want the first element that is smaller and that value won't be solution to any subsequent  $a_k$ , since  $a_i$  is smaller and further to the right).
- As soon as the value  $< a_i$ , set the answer for index  $i$  to the corresponding index in the value-index pair.
- Finally push  $(a_i, i)$  to the stack

Runtime -  $O(N)$

### 4 Sliding Window Minimum

A constant size subarray that moves from left to right is called a sliding window. To calculate and maintain a minimum of the sliding window, we use an idea similar to nearest smaller element idea, except this time we use a queue. We maintain this queue in non decreasing order. If the element being added,  $e$  is smaller than the back of the queue, we pop the back and continue to do so until the queue is empty or we reach an element smaller than  $e$ . As the window moves, we remove the elements from the front that are not part of the window anymore. The element at the front is the minimum of the window.

Runtime -  $O(n)$  because each element is added at most once and removed at most once.